ALGORITHMS AND NOTATIONS

1.1. Algorithm

A well defined set of instructions to solve a particular problem in finite no of steps is called Algorithm.

1.2 Characteristics of algorithm

Every algorithm should have the following characteristics:

1. Input
2. Output
3. Definiteness
4. Finiteness
5. Effectiveness

Input:	Every algorithm should take one or more than one input.
Output:	Every algorithm must yield at least one output.
Definiteness:	Every step of an algorithm should be very clear and unambiguous (should have a definite meaning).
Finiteness:	Every algorithm must end with a finite number of steps.
Effectiveness:	The steps present in the algorithm should be such that one can implement it.
Efficiency:	The efficiency of an algorithm can be measured by considering the time and space required.

Space Complexity: The amount of memory needed to store and to run an algorithm or program.

Time Complexity: The amount of time taken for time complexity of algorithm is given by no of steps taken by the algorithm.

1.3 Standard Notations of Algorithm

- Comments should be within [ ], e.g [an explanation related to algorithmic steps]
- Variables should be in Capital letters such as MAX, EMPLODE
- Assignment operator ← (or) = (or) **:=** and used as MAX **:**=10
- Starting the algorithm: START/BEGIN
- Stopping the algorithm: END/STOP/EXIT
- Input statement: Read/Input/Accept
- Out put statement: Write/Print/Display
- Arithmetic Operations can be specified using +, - , * , / , % (i.e. MOD)
- Relational Operations can be specified using > , < , >=, <=,!= ,=
- Logical Operations can be specified using AND , OR , NOT
- Use of Step Numbers for each statement required. E.g. Step 1, Step 2.2

Control Structure: While writing an algorithm we normally follow 3 types of logics. They are:

- Sequence logic
- Selection logic
- Iteration logic

Sequential logic: In case of sequential logic, the steps of an algorithm are executed in the same order or sequence they are available.

Ex: 1. *Write an algorithm to find simple interest for given principal amount (P), rate of interest (R) and time period (T).*

Step 1: Start
Step 2: Let P, R, T, SI
Step 3: Display "Enter principal amount, rate and time period"
Step 4: Read P, R, T
Step 5: Set SI: = (P * T * R / 100)
Step 6: Display "Simple interest is", SI
Step 7: Exit

Ex: 2. *Write an algorithm to find area of a circle.*
Step 1: Let R, AREA
Step 2: Display "Enter radius of the circle"
Step 3: Read R
Step 4: Set AREA: = 3.14 * R * R
Step 5: Display "Area of circle is", AREA

Step 6: Exit

Ex: 3. *Write an algorithm to swap two numbers without using third variable.*
Step 1: Let NUM 1, NUM 2
Step 2: Display "Enter two numbers"
Step 3: Read NUM 1, NUM 2
Step 4: Set NUM 1: = NUM 1 + NUM 2
Set NUM 2:= NUM 1 – NUM 2
Set NUM 1:= NUM 1 – NUM 2
Step 5: Display "After swapping the numbers are", NUM 1, NUM 2
Step 6: Exit

Selection Logic: In case of selection logic, only certain portion of the algorithm is selected for implementation or execution.

There are three ways how selection logic can be implemented. They are:

1. Single alternative
2. Double alternative
3. Multiple alternative

Single alternative:

Syntax:

```
If condition, Then
----------------
----------------
----------------
[End of If]
```

Ex: 1. *Write an algorithm to read a number and check whether it is positive.*

Step 1: Let NUM
Step 2: Display "Enter a number"
Step 3: Read NUM
Step 4: If NUM > 0, Then
Display "Number is positive"
[End of IF]
Step 5: Exit

Ex: 2. *Write an algorithm to read marks secured by a student and check whether grade is 'E' (If the mark is in between 80 and 90).*

Step 1: Let MARK 1
Step 2: Display "Enter the marks secured"
Step 3: Read MARK 1
Step 4: If MARK 1 >= 80 AND MARK 1 < 90, Then
Display "E Grade"
[End of If]
Step 5: Exit

Double Alternative:
Syntax:

```
If condition, Then
----------------
----------------
else
----------------
----------------
[End of If]
```

Ex: 1. *Write an algorithm to check whether a number is even or odd.*

Step 1: Let NUM
Step 2: Display "Enter a number"
Step 3: Read NUM
Step 4: If NUM mod 2 = 0, Then
Display "Number is even"
Step 5: Else
Display "Number is odd"
[End of IF]
Step 6: Exit

Ex: 2. *Write an algorithm to read two numbers and print the larger one.*
Step 1: Let NUM 1, NUM 2
Step 2: Display "Enter two numbers"
Step 3: Read NUM 1, NUM 2
Step 4: If NUM 1 > NUM 2, Then
Display "Larger number is", NUM 1
Step 5: Else
Display "Larger number is", NUM 2
[End of If]
Step 6: Exit

Multiple Alternatives:
Syntax:

```
If condition 1, Then
----------------
----------------
else if condition 2, Then
----------------
----------------
else
----------------
----------------
[End of If ]
```

Ex: 1. *Write an algorithm to accept a week number and display corresponding weekday.(i.e. 1 for Monday, 2 for Tuesday, so on)*

```
Step 1: Let WNUM
Step 2: Display "Enter a week number"
Step 3: Read WNUM
Step 4: If WNUM = 1, Then
            Display "Monday"
Step 5: Else if WNUM= 2, Then
            Display "Tuesday"
Step 6: Else if WNUM= 3, Then
            Display "Wednesday"
Step 7: Else if WNUM= 4, Then
            Display "Thursday"
Step 8: Else if WNUM= 5, Then
            Display "Friday"
Step 9: Else if WNUM= 6, Then
            Display "Saturday"
Step 10: Else
            Display "Sunday"
      [End of If ]
Step 11: Exit
```

Ex: 2. *Write an algorithm to find roots of a quadratic equation where a, b and c are coefficients and d is the discriminate.*

Step 1: Start
Step 2: Let A, B, C, D, R 1, R 2
Step 3: Display "Enter the coefficients of quadratic equation"
Step 4: Read A, B, C
Step 5: Set D: = B^2 – 4 AC
Step 6: If D>0, Then
    Step 6.1: Set R 1: = (– B + √D)/ 2 A
    Step 6.2: Set R 2: = (– B – √D)/ 2 A
    Step 6.3: Display R1, R2
Step 7: Else if D=0, Then
    Step 7.1: Set R1: = –B/ 2A
    Step 7.2: Display "Unique solution", R1
Step 8: Else
    Step 8.1: Display "Roots are imaginary"
    [End of If]
Step 9: Exit

Iteration Logic (Repetition) : Iteration logic is used when certain steps of an algorithm needs to be executed repeatedly. Iteration logic is otherwise called as concept of loops.
Iteration logic can be implemented by using

1. Repeat While
2. Repeat For

Repeat While Loop:

Syntax:
```
Repeat While condition
----------------
----------------
----------------
[End of While]
```

Ex: 1. *Write an algorithm to print natural numbers up to 10.*

Step 1: Start
Step 2: Let X
Step 3: Set X: = 1
Step 4: Repeat While X <= 10
    Step 4.1: Display X
    Step 4.2: Set X: = X + 1
    [End of While]

Step 5: Exit

Ex: 2. *Write an algorithm to read a number and find sum of digits of it.*

Step 1: Start
Step 2: Let NUM, DIGIT, SUM
Step 3: Set SUM: = 0
Step 4: Display "Enter the number"
Step 5: Read NUM
Step 6: Repeat While NUM ≠ 0
    Step 6.1: Set DIGIT: = NUM mod 10
    Step 6.2: Set SUM: = SUM + DIGIT
    Step 6.3: Set NUM: = NUM / 10
    [End of While]
Step 7: Display "Sum of digits is", SUM
Step 8: Exit

Repeat For Loop:

Syntax: Repeat for I = R to S by T

[End of For]

[Where R is the initial value, S is the final value and T is the change in value]

Ex: 1. *Write an algorithm to generate Fibonacci series up to 20.*

Step 1: Start
Step 2: Let A, B, C, K
Step 3: Set A: =0, B: = 1, C: = A + B
Step 4: Display A, B
Step 5: Repeat For K=3 to 20 by 1
    Step 5.1: Display C
    Step 5.2: Set A: = B
    Step 5.3: Set B: = C
    Step 5.4: Set C: = A + B
    [End of For]
Step 6: Exit

Ex: 2. *Write an algorithm to find factorial of a positive number.*

Step 1: Start
Step 2: Let N, K, F
Step 3: Set F: = 1
Step 4: Display "Enter a number"
Step 5: Read N
Step 6: Repeat For K=1 to N by 1
    Step 6.1: Set F: = F * K
    [End of For]
Step 7: Display "Factorial of the number is", F
Step 8: Exit

ARRAYS AND STRUCTURES

2.1 ARRAY

Array: It is a collection of ***similar*** data elements present in continuous memory locations and referred by a unique name.

Defining Array:
Syntax:

Data type array name[size];

e.g., int A[5]; int A[ ] ={1, 2, 3, 4,5};

2.2 Types of Arrays:

- Single dimensional array
- Multi dimensional array.

Finding Address of an element in an 1-D Array

Assume an array A of size 10. i.e. A[10]

100	104	108	112	116	120	124	128	132	136
5	9	2	11	3	8	7	6	4	1
0	1	2	3	4	5	6	7	8	9

LB(lower bound) K UB(upper bound)

e.g. Base Address (B) = 100
LB = 0, UB=9
Size/ Width of each element W=4 byte
K is at position 5
Address of an element with index K:

Loc (A [K]) = B + W (K- LB)
= 100 + 4 *(5 - 0)
= 120

So Address of A [5] is 120

2.3 Operations on One dimensional array

Creation: Create or entering elements into Array
Traversal: Processing each element
Search: Searching for an element
Insertion: Inserting an element at required position.
Deletion: Deletion of an element
Sorting: Arranging in ascending or descending order
Merging: Two ways

1. Join/concatenate two arrays into 3rd Array and then sort.
2. Merge directly into 3rd Array where two arrays are already sorted.

2.3.1 Creation of an array:

Algorithm:

CREATE (LA, LB, UB)

Step 1: Let LA [10], LB, UB, K
Step 2: LB**:** =1, UB**:** =10,
Step 3: Display "Enter 10 Elements"
Step 4: Repeat, for K**:**=LB to UB incremented by 1
INPUT LA [K]
[End of For]
Step 5: Exit.

2.3.2 Traversing an array:

Algorithm:

TRAVERSE (LA, LB, UB)

Step 1: Repeat for K=LB to UB by 1
Apply PROCESS to LA [K]
[End of For]

Step 2: Exit.

- Here PROCESS can be visiting each element and do calculation on it.
- In 'C' language LB is 0 by default and UB is Last position of Array.

2.3.3 Searching for an element in an array:

Algorithm:

SEARCH (LA, LB, UB, ITEM)

```
Step 1:     Let K, Set K: =LB
Step 2:     Repeat, while (K<=UB)
        Step 2.1: If LA [K] =ITEM, Then
                      Display "Item Found at", K
                [End of IF]
        Step 2.2: Set K: =K+1
        [End of While]
Step 3:     Exit
```

2.3.4 Insertion of new element in an array

Algorithm:

INSERTION (LA, LB, UB, ITEM, POS)

```
Step 1:     Let K
Step 2:     Repeat, for K=UB to POS, decreasing by 1
            LA[K+1]: =LA[K]
        [End of For]
Step 3:     LA [POS]: =ITEM
Step 4:     Set UB: = UB + 1
Step 5:     EXIT
```

3.3.5 Deletion of an element from an array

Algorithm:

DELETION (LA, LB, UB, POS)

```
Step 1: Let K
Step 2:     Repeat for K: =POS to UB incremented by 1
            LA[K]: = LA[K+1]
        [End of For]
        UB:=UB-1
```

Step 3: Exit

2.3.6 Sorting the elements of an array

Algorithm:

SORTING (LA, LB, UB)

```
Step 1:      Let I, J, K, T
Step 2:      Set I:=1, J:=2
Step 3:      Repeat for K: =UB-1 to 1 by decrementing by 1
   Step3.1:  Repeat for I:=1, J:=2 to K by 1
      Step 3.1.1:  If LA[I]> LA[J], then
                        T :=LA[I]
                        LA[I] :=LA[J]
                        LA[J] := T
                   [End of IF]
             [End of For]
      [End of For]
Step 4:      Exit.
```

2.3.7 Merging Of Two Arrays

Algorithm :(Method – I)

[Assume two arrays are not in sorted manner. Here there are three arrays A[5], B[5] and C[10]. A[5] and B[5] already contains numbers in unsorted manner]

MERGE (A,B,C,LB1,UB1,LB2,UB2,LB3,UB3)

[*LB1 and UB1 are lower and upper bounds of Array A, LB2 and UB2 are lower and upper bounds of Array B, LB3 and UB3 are lower and upper bounds of Resultant Array C*]

```
Step 1:      Let I, K
Step 2: Set K: =LB3
Step 3:      Repeat for I:=LB1 to UB1 incrementing by 1
             C[K]:=A[I]
             K:=K+1
      [End of For]
Step 4:      Repeat for I=LB2 to UB2 incrementing by 1
             C[K]:=B[I]
```

```
                K:=K+1
        [End of For]
Step 4:         Call SORTING (C, LB3, UB3)
Step 5:         Exit.
```

2.3.7 *Merging Algorithm where given two arrays are already in sorted manner*

<u>Algorithm: (Method – II)</u>

MERGE (A,B,C, LB1, UB1, LB2, UB2, LB3, UB3)

```
Step 1:         Let I, J, K
Step 2:         Set I:=LB1, J:=LB2, K:=LB3
Step 3:         Repeat While I<=UB1 AND J<=UB2
    Step3.1:    If A[I]<B[J], then
                        C[K]:=A[I]
                        K:=K+1
                        I:=I+1
                Else
                        C [K]:=B [J]
                        J: =J+1
                        K: =K+1
                [End of IF]
        [End of While]
Step 4:         Repeat While I<=UB1
                C [K]:=A [I]
                K: =K+1, I: =I+1
        [End of While]
Step 5:         Repeat, while J<=UB2
                C[K]:=B[J]
                 K: =K+1, J: =J+1
        [End of While]
Step 6:         Exit
```

2.4 MULTI DIMENSIONAL ARRAYS:

While discussing Multi-dimensional array, we will concentrate on the concepts related to 2-D array (i.e. Matrix).In general, a two dimensional array is row and column arrangement of elements.

A multi-dimensional array can be represented by an equivalent one dimensional array. A two dimensional array consists of number of rows and columns. In other words, a two dimensional array can be referred as *array of arrays.*

For Example, A [3] [4] is a two dimensional array with 3 rows and 4 columns. In other words, we can say there are 3 one dimensional arrays having 4 elements each.

7	9	20	1
11	5	2	17
13	15	21	19

2.4.1 Memory Representation of 2-D Array:

Any multidimensional array (e.g 2 – D array) can be always represented in the form of 1-D array in the memory. There are two ways how a 2-D array is stored in the memory. They are:

- Row Major Order
- Col. Major Order.

2.4.1.1 Row – Major Order

In row-major order, elements of 2-D array are stored on a row-by-row basis beginning from first row. i.e. the majority is given to the rows present in the matrix.

For example, the above matrix A[3][4] can be represented in row-major order in the following format.

7	11	13	1	11	5	2	17	13	15	21	19

A[0][0] A[0][1] A[0][2] A[0][3] A[1][0] A[1][1] A[1][2] A[1][3] A[2][0] A[2][1] A[2][2] A[2][3]

(Row – Major Order Representation)

Address of an element when represented in row-major order can be calculated by using the following formula.

Address of A[I] [J] = B + W * [(N * (I- LB_R)) + (J- LB_C)]

e.g. Address of A[1][3]

= 100 + 2 * [(4*(1-0))+ (3-0)]
= 100 + 2 *[(4*1) +3]
= 100 +2 * 7
= 114

In 'C' Language (as LB_R = 0 and LB_C = 0), the above formula can also be represented as per the following.

Address A[I][J] = B + W * (N*I+J)

2.4.1.2 Column – Major Order

In column-major order, elements of 2-D array are stored on a column-by-column basis beginning from first column. i.e. the majority is given to the columns present in the matrix.

For example, the above matrix A [3] [4] can be represented in column-major order in the following format.

7	9	20	9	5	15	20	2	21	1	17	19

A[0][0] A[1][0] A[2][0] A[0][1] A[1][1] A[2][1] A[0][2] A[1][2] A[2][2] A[0][3] A[1][3] A[2][3]

(Column – Major Order Representation)

Address of an element when represented in column-major order can be calculated by using the following formula.

Address of A[I] [J] = B + W * [(I- LB_R) + M* (J- LB_C)]

e.g. Address of A[1][3] = 100 + 2 * [(1-0) + 3 *(3-0)]
= 100 + 2* [1 + 9]
= 100 + 20
= 120

In 'C' Language (When $LB_R = 0$ and $LB_C = 0$)

Address of A[I] [J]= B + W * [I + M * J]

Row Major Order	**Address**	**Col. Major Order**	**Address**
A[0][0]	100	A[0][0]	100
A[0][1]	102	A[1][0]	102
A[0][2]	104	A[2][0]	104
A[0][3]	106	A[0][1]	106
A[1][0]	108	A[1][1]	108
A[1][1]	110	A[2][1]	110
A[1][2]	112	A[0][2]	112
A[1][3]	114	A[1][2]	114
A[2][0]	116	A[2][2]	116
A[2][1]	118	A[0][3]	118
A[2][2]	120	A[1][3]	120
A[2][3]	122	A[2][3]	122

2.5 SPARSE MATRIX

Matrices with relatively high proportion of zero or null entries are called sparse matrices.

When matrices are sparse, then much space and computing time could be saved if the nonzero entries were stored explicitly i.e. ignoring the zero entries the processing time and space can be minimized in the sparse matrices.

0	0	0	24	0	0	0
0	0	0	0	0	5	0
0	0	0	0	0	0	0
0	0	0	0	9	0	0
0	0	0	0	18	0	0
0	0	0	0	8	0	0

In the above matrix we have 6 rows and 7 columns. There are 5 nonzero entries out of 42 entries. It requires an alternate form to represent the matrix without considering the null entries.

The alternate data structure that we consider to represent a sparse matrix is a triplet. The triplet is a two dimensional array having t+1 rows and 3 columns. Where, t is the total number of nonzero entries.

The first row of the triplet contains number of rows, columns and nonzero entries available in the matrix in its 1st , 2nd and 3rd column respectively. Second row onwards it contains the row subscript, column subscript and the value of the nonzero entry in its 1st, 2nd and 3rd column respectively.

Let us represent the above matrix in the following triplet of 6 rows and 3 columns

6	7	5
1	4	24
2	6	5
4	5	9
5	5	18
6	5	8

The above triplet contains only nonzero details by reducing the space for null entries.

The below algorithm performs operations on:

- To store the nonzero entries of a sparse matrix into alternate triplet representation

- Displaying the triplet representation
- Transpose of the triplet representation
- Displaying the transpose of sparse matrix using triplet

2.5.1 Algorithm for operation on a sparse matrix

```
Step 1:     Let A[10][10], SP[50][3], TP[50][3], I, J, M, N,NZ
Step 2:     NZ:=0
Step 3:     Display "Enter the row and column size"
Step 4:     Input M, N
Step 5:     Repeat, for I:=0 to M-1  incremented by 1
      Step5.1: Repeat for J:=0 to N-1 incremented by 1
                  Input A[I][J]
                  If A[I][J] != 0, then
                        NZ =NZ+1
                  [End of if]
            [End of for – step5.1]
      [End of for – step5]
Step 6:     If NZ>= (M*N)/2, then
            Display  " Matrix is not sparse"
Step 7:     else
            Display  " Matrix is sparse"

      [Calling the procedure to store Non-Zero elements
      from Matrix A into alternate Triplet Matrix SP]
      Step 7.1: CALL ALTERNATE(A,SP,M,N,NZ)

      [Calling the procedure to display original matrix using Triplet Matrix SP]
      Step 7.2: CALL DISPLAY_ALTERNATE (SP, M,N)
      [End of if – step 6]

Step 8:     Exit
```

2.5.1.1 ALGORITHM TO REPRESENT A SPARSE MATRIX IN TRIPLET FORMAT

```
ALTERNATE (A, SP, M, N, NZ)
Step 1:     Let I, J, K: =1
Step 2:     SP[0][0]:=M,SP[0][1]:=N,SP[0][2]:=NZ
```

```
Step 3:         Repeat For I:=0 to M-1 incremented by 1
        Step 3.1: Repeat for J:=0 to N-1 incremented by 1
                        If  A[I][I]=0 then
                                SP[K][0]:=I
                                SP[K][1]:=J
                                SP[K][2]:=A[I][J]
                                K:=K+1
                        [End of if]
                [End of for – Step3.1]
        [End of for – Step3]
Step 4:         Repeat, For K:=0 to NZ incremented by 1
                Display SP[K][0],SP[K][1],SP[K][2]
        [End of For]
Step 5:         Exit.
```

2.5.1.2 ALGORITHM TO DISPLAY ORIGINAL MATRIX USING TRIPLET (SP)

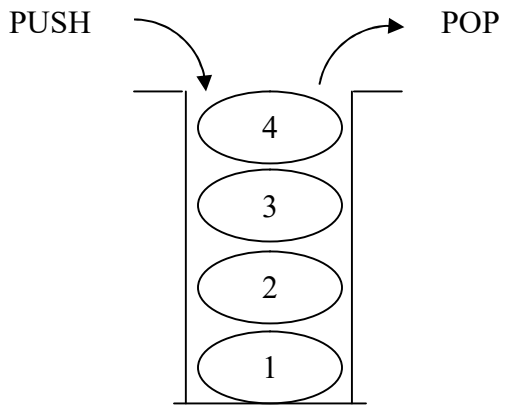
```
DISPLAY_ALTERNATE (SP, M,N)
Step 1:         Let I, J, K
Step 2:         K :=1
Step 3:         Repeat For I :=0 to M-1 increased by 1
        Step3.1: Repeat For J:= 0 to N-1 increased by 1
                        If (SP[K][0] =I AND SP[K][1]=J), then
                                Display SP[K][2]
                                K:=K+1
                        Else
                                Display '0'
                        [End of IF]
                [End of FOR – Step 3.1]
        [End of FOR – Step 3]
Step 4:         Exit.
```

STACK AND ITS APPLICATIONS

3.1 STACK

It is a non primitive linear data structure. It is an ordered list, where insertion & deletion can be done at one end only.

e.g. Stack of plates one above the other



Here the last inserted element can be taken out first and this concept is called LIFO (Last In First Out).

Characteristics:

1) Deletion of an element can be done only from top of the stack
2) Insertion of an element can be done only on top of the stack

3.2 Operations on Stack:

1) **PUSH**: Insertion of an element into the stack.
2) **POP**: Deletion of an element from the top of the stack.

3.2.1 PUSH Operation:

In the stack, TOP is the key which always identifies the recent element pushed. While pushing a new element into the stack, one must test whether there is a room in the stack or not. If room is available then the element can be pushed (i.e. the TOP identifies the next available room in the stack and the element is stored in the room).

OVERFLOW situation occurs when TOP identifies the last available room of the stack (i.e. stack is full), and one performs the PUSH operation.

3.2.2 POP operation

While performing the POP operation, deletion of an element can be done from the room that is identified by TOP and TOP will identify the second topmost room of the Stack.

UNDERFLOW situation occurs, when no element is present in the stack (i.e. when TOP identifies NULL) and POP operation is performed on it.

3.3 Memory Representation of Stack

There are two ways to represent the stack in memory. They are

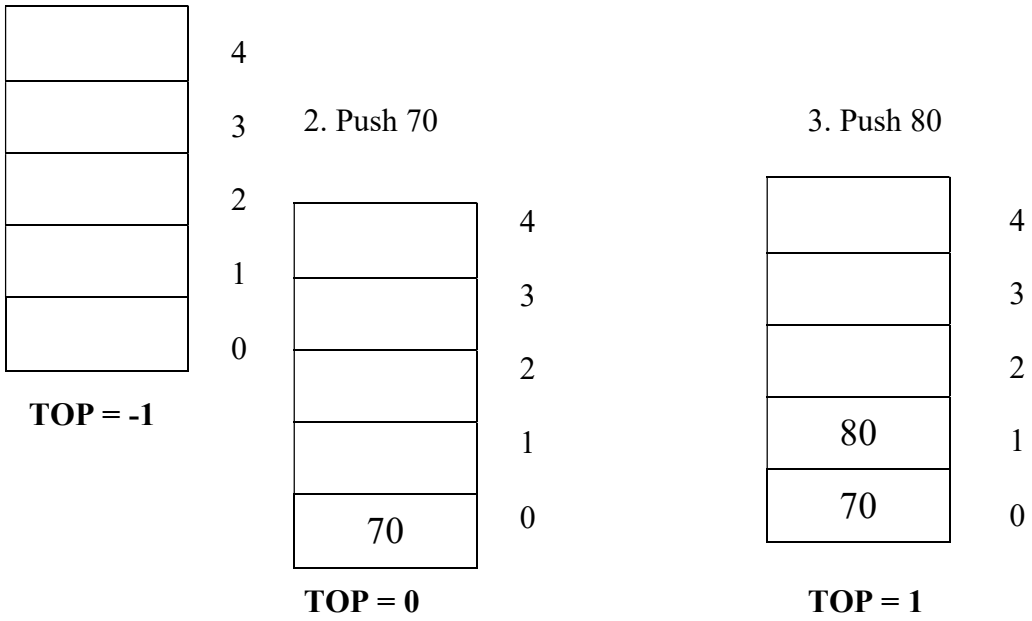
1) using array (static memory)
2) linked list (dynamic memory)

3.3.1 Array representation of stack

A one dimensional array can be considered for representing a stack in memory. While representing a stack in the form of an array we take a variable TOP that identifies topmost element of the stack.

For example, assume an array S [5] is considered to represent a stack, TOP is a variable contains the index of topmost element. The following example describes the process of push and pop operations and illustrates the change in the TOP value.

1. Initially when the stack is empty. **TOP = – 1**



4. Pop an element

	4
	3
	2
	1
70	0 **TOP = 0**

5. Push 90, 80, 30

	4
30	3 **TOP = 3**
80	2
90	1
70	0

6. Pop an element

	4
	3
80	2 **TOP = 2**
90	1
70	0

From the above example we see that, after each push operation, value of TOP is increased by 1 and after each pop operation, the value of TOP is decreased by 1.

Condition of Overflow:

When TOP= Size – 1 (Where size is the maximum number of elements that can be hold by the array that represents the stack in memory), if PUSH operation is performed.

Condition of Underflow:

When TOP = – 1 (i.e. When stack is empty), if POP operation is performed.

3.3.2 Linked representation of stack

The concept of dynamic memory allocation is implemented by using linked list to represent stack in linked format.(Refer Chapter 5.2 of Module 2)

3.4 Algorithm for PUSH Operation

[Let stack [size] is an Array, Top identifies the topmost element. ITEM is the element to be pushed onto the stack]

PUSH (STACK, TOP, SIZE, ITEM)

```
Step 1: If (TOP = SIZE – 1), then
            Display "Overflow"
            EXIT
Step 2: Else
            TOP:=TOP+1
                STACK[TOP]:=ITEM
      [End of IF]

Step3: Exit.
```

3.5 Algorithm for POP Operation

<u>Algorithm:</u>

```
Step 1: Let ITEM
Step 2: If (TOP<0), Then
            Display "Underflow"
            EXIT
Step 3: else
            ITEM: = STACK [TOP]
            Display "Popped Item is", ITEM
            Top: =Top – 1
      [End of IF]
Step 4: Exit.
```

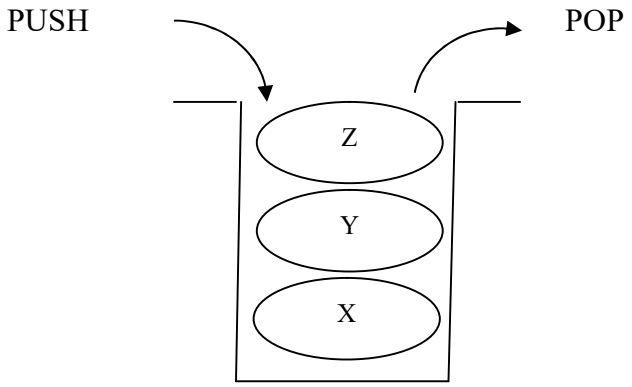
3.6 Applications of stack

3.6.1 Keeping track of function calls

The processing of sub programs calls and their return values are an example of stack structure.
For Example:

Here the main( ) calls function Fun1( ). The function Fun1( ) calls function Fun2( ). The function Fun2( ) calls function Fun3( ). Let X , Y and Z are the addresses of function Fun1( ), Fun2( ) and Fun3( ) respectively. During the function call the addresses are stored (or Pushed) onto the stack one by one. And then the inner most function's address which remains at the Top of the Stack is popped and processed and so on.

```
main( )          Fun1( )          Fun2( )          Fun3( )
{                {                {                {
:                :                :                :
R1=Fun1( );      R2=Fun2( );      R3=Fun3( );      :
:                :                :                :
}                }                }                }
```



3.6. 2 Recursion (Self Reference)

Recursion can be defined as the process of a function by which it can call itself. The function which calls itself again and again either directly or indirectly is known as recursive function.

The recursive function must have the following properties:

1. There must be certain criteria called base criteria, for which the function does not call itself.
2. Each time the function calls itself directly or indirectly, it must be closer to the base criteria.

A recursive function with these two properties is said to be well defined.

Depth of Recursion:

Suppose P is a recursive function. During the execution of a program which contains P, it associate level number with each execution of P, the original execution of P is assigned level 1. Each time P is executed because of recursive call, its level is one more than the level of execution that had made recursive call.

The depth of recursive function P with given set of arguments refers to the maximum level number of P during its execution.

Example of Recursive function:

```
Factorial of a positive number N
Factorial( N )
Step 1: if (N = 0 ), then
                Return 1
Step 2:         else
                Return ( N * Factorial ( N-1 ) )
        [ End of IF ]
Step 3: Exit
```

Assume N = 5, the progress of execution of the above recursive function is as follow:

```
Factorial ( 5 )     =     5 * Factorial ( 4 )
                =     5 * (4 * ( Factorial ( 3 ) )
                =     5 * ( 4 * ( 3 * ( Factorial ( 2 ) ) )
                =     5 * ( 4 * ( 3 * ( 2 * ( Factorial ( 1 ) ) ) )
                =     5 * ( 4 * ( 3 * ( 2 * ( 1 * ( Factorial ( 0 ) ) ) ) )
                =     5 * ( 4 * ( 3 * ( 2 * ( 1 * 1 ) ) ) )
```

= 5 * (4 * (3 * (2 * 1)))
= 5 * (4 * (3 * 2))
= 5 * (4 * 6)
= 5 * 24
= 120

There are three methods for solving a problem :

1. A recursive solution
2. An iterative solution
3. An analytical solution

There are no set of predefined rules to select a particular method. So we must consider the following issues.

1. processing time taken
2. computer memory used
3. time taken to develop a program
4. time taken to debug a program
5. time to maintain a program

General method for removing Recursion: In a recursion implementation, the storage areas for sub programs are kept in a stack. Without recursion one permanent storage area is reserved for each sub program, so that an attempt to make a recursive call would change the values in the storage area, there by destroying the ability of the outer call to complete its work and return properly.

Divide and Conquer:

Divide and Conquer can be defined generally as the method of solving a problem by dividing into two or more sub problems, each of which is similar to the original problem in nature, but smaller in size. Solution to the sub programs are then obtained separately and combined to produce the solution of the original problem.

3.6. 3. CONVERSION & EVALUATION OF ARITHMETIC EXPRESSIONS

3.6.3.1 Arithmetic expression notations

In any arithmetic expression, each operator is placed in between two operands of it. (i.e. mathematical representation)
This way is called as : Infix Expression. e.g. A+B.

Apart from usual mathematical representation of an arithmetic expression, an expression can also be represented in the following two ways:

1. Polish or Prefix Notation
2. Reverse Polish or Postfix Notation

Polish or Prefix Notation

The notation in which the operator symbol is placed before its operands, is referred as polish or prefix notation. For Example: +AB.

Reverse Polish or Postfix Notation

The notation, in which the operator symbol is placed after its operands, is referred as reverse polish or postfix notation. For Example: AB+

Advantage of Prefix & Post fix over Infix Notation is no need of parenthesis in the expression.

3.6.3.2 Mathematical Procedure for conversion of infix expression to post fix / prefix expression

Standard Arithmetic Operators and Precedence Levels:

^ (exponential)	Higher Level
* , / , %	Middle Level
+ , -	Lower Level

1. Parenthesize the expression starting from left to right.
2. During parenthesizing the expression, the operands associated with operators having higher precedence are first parenthesized.
3. The sub expression which is parenthesized is converted into postfix or prefix and should be treated as single operand.
4. Once the expression is converted into postfix or prefix, remove the parenthesis.

Example : Convert the following infix expression into corresponding postfix and prefix expression.

(A + B * (C – D) / E)

<u>Infix to Post Fix</u>

e.g. (A + B * (C – D) / E)
= (A + B * [CD-] / E)
= (A + [BCD-*] / E)
= (A + [BCD-*E/])
= ABCD-*E/+

<u>Infix to Prefix:</u>

e.g. (A + B * (C – D) / E)
= (A + B * [-CD] / E)
= (A + [*B-CD] / E)
= (A + [/*B-CDE])
= +A/*B-CDE

Example : Convert the following infix notation into prefix and postfix notation.

1) (A/(B^C)+D)
2) ((A*B+(C/D))-F)

3.6.3.3 Importance of Postfix Expression:

Although human beings are quite used to work with mathematical expression i.e. INFIX notation, which is rather complex as using this notation one has to remember a set of rules. The rules include BODMAS and ASSOCIATIVITY.

In case of POSTFIX notation, the expression, which is easier to work or evaluate the expression as compared to the INFIX expression. In a POSTFIX expression, operands appear before the operators, there is no need to follow the operator precedence and any other rules.

Actually, the processor represents the mathematical expression in postfix expression and uses it for evaluation. To do this it implements the concept of stack.

3.6.3.4 Conversion of an INFIX expression into POSTFIX expression using Stack:

Algorithm for converting from INFIX to POSTFIX

[Assume Q is an Infix expression and P is the corresponding Postfix notation.]

Step 1: PUSH '(' onto STACK and Add ')' at the end of Q

Step 2: Repeatedly scan from Q Until STACK is Empty

Step 2.1: If Q[I] is an operand, Then Add it to P[J]
Step 2.2: Else if Q[I] is '(', Then PUSH Q[I] into STACK
Step 2.3: Else if Q[I] is an operator , Then
Step 2.3.1: While STACK[TOP] is an operator and has higher or equal priority compared to Q[I]
Pop operators from STACK and add to P[J]
[End Of While]
Step 2.3.2: Push operator Q [I] onto STACK
Step 2.4: Else if Q[I] is an ')', Then
Step 2.4.1: Repeatedly pop operators from STACK and add to P[J] until '(' is encountered
Step 2.4.2: Remove '(' from STACK
[End Of IF]
[End Of Loop Step – 2]
Step 3: Exit

Procedure of Conversion

e.g. Q = (B * C - (D / E ^ F))

Symbol Scanned from Q	Stack	Postfix Expression P
(	(	
B	(	B
*	(*	B
C	(*	BC
-	(-	BC*
(	(- (	BC*

D	(- (	BC*D
/	(- (/	BC*D
E	(- (/	BC * DE
^	(- (/ ^	BC * DE
F	(- (/ ^	BC * DEF
)	(- (/ ^	BC * DEF ^/
)	(-	BC * DEF ^ / -

3.6.3.5 EVALUATION OF POST FIX EXPRESSION USING STACK

[Assume P is a post fix expression]

Step 1: Add ')' at the end of P
Step 2: Repeatedly scan from P until ')' encountered
Step 2.1: if P[I] is an operand, then
Push the operand onto STACK
Step 2.2: else If P[I] is an op⊗ator then
Step 2.2.1: Pop two elements from STACK
(1st element is A and 2nd element is B)
Step 2.2.2: Evaluate B⊗ A
Step 2.2.3: Push result back to STACK
[End of IF]
[End of loop – Step 2]
Step 3: VALUE: = STACK [TOP]
Step 4: Display VALUE
Step 5: Exit.

For Example:

Evaluate the following Postfix Expression using Stack:

P = B C * D E F ^ / -
Where B = 5, C = 6, D = 24, E = 2, F = 3

So, P = 5, 6, * 24, 2, 3, ^, /, -

Symbol Scanned from Post Fix Notation	Stack	A	B	B (Op) A
5	5			
6	5,6			
*	30	6	5	5 * 6 =30
24	30, 24			
2	30,24,2			
3	30,24,2,3			
^	30,24,8	3	2	2 ^ 3 = 8
/	30,3	8	24	24 / 8 = 3
-	27	3	30	30 – 3 = 27

4.7.3.6 Conversion of infix expression to prefix expression using STACK:

[Assume Q is an infix expression. Consider there are two stacks S1 and S2 exist. The following algorithm converts the infix expression Q into its equivalent Prefix notation.]

<u>Algorithm:</u>

Step 1: Add left parenthesis '(' at the beginning of the expression Q
Step 2: PUSH ')' onto Stack S1
Step 3: Repeatedly Scan from Q in right to left order, Until Stack S1 is Empty
    Step 3.1: If Q[I] is an operand, Then PUSH it onto Stack S2
    Step 3.2: Else if Q [I] is ')', Then PUSH it onto Stack S1
    Step 3.3: Else if Q [I] is an operator (OP) , Then
        Step 3.3.1: Set X = POP (S1)
        Step 3.3.2: Repeat while X is an Operator AND (Precedence(X) > Precedence(OP))
            PUSH (X) onto Stack S2
            Set X = POP (S1)
        [End of While – Step 3.3.2]
        Step 3.3.3: PUSH (X) onto Stack S1
        Step 3.3.4: PUSH (OP) onto Stack S1

Step 3.4: Else if Q [I] is '(' , Then
Step 3.4.1: Set X = POP(S1)
Step 3.4.2: Repeat, While(X != ')') [Until right parenthesis found]
Step 3.4.2.1: PUSH (X) onto Stack S2
Step 3.4.2.2: Set X = POP(S1)
[End of While – Step 3.4.2]
[End of IF – Step 3.1]
[End of Loop – Step 3]

Step 4: Repeat, While Stack S2 is not Empty
Step 4.1: Set X = POP (S2)
Step 4.2: Display X
[End of While]

Step 5: Exit

<u>Procedure of Conversion</u>

e.g. Q = (A + B * C * (M * N ^ P + T) – G + H

Symbol Scanned from Q	Stack S1	Stack S2
H	)	H
+	)+	H
G	)+	HG
-	)+-	HG
)	)+-)	HG
T	)+-)	HGT
+	)+-)+	HGT
P	)+-)+	HGTP
^	)+-)+^	HGTP
N	)+-)+^	HGTPN
*	)+-)+*	HGTPN^
M	)+-)+*	HGTPN^M
(	)+-	HGTPN^M*+
*	)+-*	HGTPN^M*+
C	)+-*	HGTPN^M*+C
*	)+-**	HGTPN^M*+C

B	)+-**	HGTPN^M*+CB
+	)+-+	HGTPN^M*+CB**
A	)+-+	HGTPN^M*+CB**A
(		HGTPN^M*+CB**A+-+

So the Prefix Notation We can get by popping all the symbols from Stack S2.
i.e. +-+A**BC+*M^NPTGH

4.7.3.7 Evaluation of Prefix Expression:

[Assume P is a Prefix Expression]

Step 1: Add '('at the beginning of the prefix expression
Step 2: Repeatedly scan from P in right to left order until '('encountered
    Step 2.1: If P [I] is an operand, then
        PUSH the operand onto STACK
    Step 2.2: Else if P [I] is operator (OP), then
        Step 2.2.1: Pop two elements from STACK
        (1st element is A and 2nd element is B)
        Step 2.2.2: Evaluate A (OP) B
        Step 2.2.3: Push result back to STACK
        [End of if]
    [End of loop – Step 2]
Step 3: VALUE: = STACK [TOP]
Step 4: Display VALUE
Step 5: Exit.

For Example:

Evaluate the following Prefix Expression using stack:

P = (- , * , 3 , + , 16 , 2 , / , 12 , 6

Symbol Scanned from Prefix Expression in Right to Left Order	Stack	A	B	A (OP) B
6	6			
12	6 , 12			
/	2	12	6	12 / 6 = 2
2	2 , 2			
16	2 , 2 , 16			
+	2 , 18	16	2	16 + 2 = 18
3	2 , 18 , 3			
*	2 , 54	3	18	3 * 18 = 54
-	52	54	2	54 – 2 = 52

Finally the value in the Stack is 52.

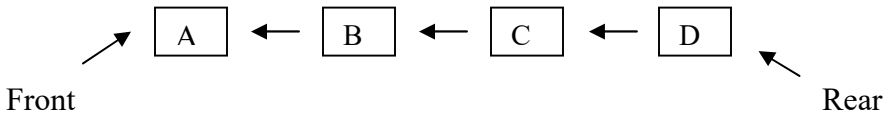
QUEUES

5.1 QUEUE

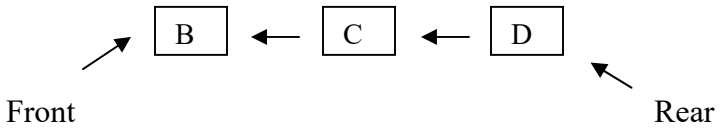
A Queue is a linear list of elements in which deletion is possible at one end called FRONT and insertion is possible at other end called REAR.

e.g. people in a queue at the Bus Stop.

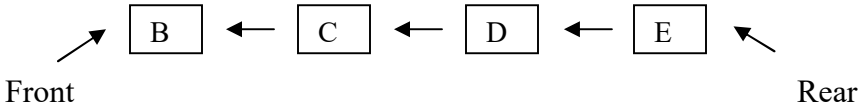
e.g. Consider below a list of elements exist



After deleting an element from Front end



After inserting element “E” at Rear end



Here the first element available in the list i.e. at FRONT end can be taken out first and this concept is called FIFO (First In First Out).

General Characteristics

1) **Deletion** of an element can be done from FRONT end
2) **Insertion** of an element can be done at REAR end

5.2 Types of Queues are:

1) Linear Queue
2) Circular Queue
3) Double Ended Queue or DEQUE
4) Priority Queue

5.3 Memory Representation of Queue:

There are two ways to represent the Queue in memory. They are

1) using array (Static Implementation)
2) linked list (Dynamic Implementation)

5.3.1 Array representation of queue

The concept of static memory allocation can be implemented by using one dimensional array to represent a queue in memory. We consider two variables FRONT and REAR which holds the index of front and rear end of the queue.

5.3.2 Linked representation of stack

The concept of dynamic memory allocation is implemented by using linked list to represent queue in linked format.(Refer Chapter 5.3 of Module 2)

5.4 LINEAR QUEUE

Insertion Operation:

In the Queue, while inserting a new element, one must test whether there is a room available at REAR-End or not. If room is available then the element can be inserted (i.e. the REAR identifies the next available room in the queue and the element is stored in the room).

OVERFLOW situation occurs when REAR identifies the last available room of the queue (i.e. queue is full), and one performs the insertion operation.

Deletion operation:

Deletion of an element can be done from the room that is identified by FRONT and FRONT will identify the next room of the queue.

UNDERFLOW situation occurs, when no element is present in the queue(i.e. when FRONT or REAR identifies NULL) and deletion operation is performed on it.

For Example:
Let Q [N] is an array that represents a queue in the memory and at the beginning it is empty.
i.e. Front = Rear = NULL or – 1

FRONT = -1

REAR = -1

0	1	2	3	4	5	6	7	8	9
							N		

1. ***Insert 4 elements*** ***A, B, C, D***

FRONT = 0

REAR = 3

A	B	C	D						
0	1	2	3	4	5	6	7	8	9

2. ***Delete 2 elements***

FRONT = 2

REAR = 3

		C	D						
0	1	2	3	4	5	6	7	8	9

3. ***Insert 3 elements*** ***E, F, G***

FRONT = 2

REAR = 6

		C	D	E	F	G			
0	1	2	3	4	5	6	7	8	9

From the above example, it is clear that:

1. For each insertion in the queue the value of **REAR** is incremented by 1 i.e. **REAR=REAR +1**
2. For each deletion in the queue the value of **FRONT** is incremented by 1 i.e. **FRONT=FRONT + 1**

NOTE:

1) So after N elements inserted onto the Queue, REAR will identify (N-1)th position (i.e. Queue becomes full).
2) Even though the memory locations are vacant at FRONT end, (When REAR identifies N -1 position), we cannot insert a new element.
3) When FRONT=REAR=NULL (i.e. queue is empty), we cannot perform deletion operation.
4) When FRONT=REAR only one Element is present in Queue.
5) When FRONT is at N-1, means Queue having one element. After deletion, FRONT need to be reset to NULL.

5.4.1 ALGORITHM FOR INSERTION INTO A LINEAR QUEUE:

[Assume QUEUE [N] is an array which represents a queue, FRONT and REAR identifies the index of front and rear end respectively. ITEM is the element to be inserted into the QUEUE]

QINSERT (QUEUE, FRONT, REAR, N, ITEM)

```
Step 1:         If REAR = N-1, then
            Display "Over Flow or Queue Full "
            Exit.
Step 2:         Else if FRONT=NULL, then
            FRONT: =0
            REAR: =0
Step 3: Else
            REAR: =REAR+1
      [End of IF]
Step 4:         QUEUE [REAR]:=ITEM
Step 5Exit
```

5.4.2 ALGORITHM FOR DELETION IN A LINEAR QUEUE:

[Assume QUEUE [N] is an array which represents a queue, FRONT and REAR identifies the index of front and rear end respectively. ITEM holds the element to be deleted from the QUEUE]

QDELETE (QUEUE, FRONT, REAR, N)

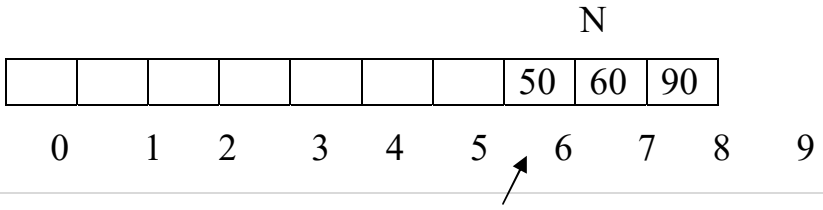
Step 1: Let ITEM
Step 2: If FRONT=NULL, Then [When QUEUE has no elements]
Display "Underflow (or) Queue Empty"
Exit
[End of IF]
Step 3: ITEM: =QUEUE [FRONT]
Step 4: Display "Deleted Item is", ITEM
Step 5: Else if FRONT=REAR, Then [When QUEUE contains only one element]
FRONT: =NULL
REAR: = NULL
Step 5: Else
FRONT: =FRONT+1
Step 6: Exit.

5.5 CIRCULAR QUEUE

It is a queue of circular nature in which it is possible to insert a new element at the first location when the last location of the queue is filled.

In other words, if we have an array Q [N] representing a circular queue, then after inserting an element at $(N-1)^{th}$ location of the array, the next element will be inserted at the very first location (i.e. location with index 0).

5.5.1 Drawback of linear queue:

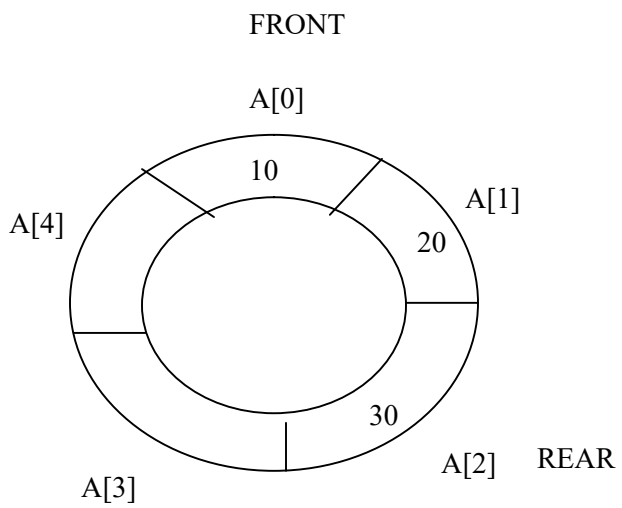


Front Rear

In case of linear queue, if REAR is at N-1 then queue is considered as full. Here even though memory locations are vacant at beginning still we can not use them, as for each insertion rear is incremented, and here REAR goes out of bounds. This drawback can be overcome by circular queue representation.

5.5.2 Circular queue representation:

For example:
Let an array A [5] is maintained in the form of circular queue.



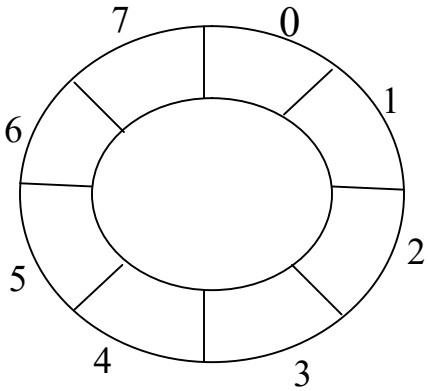
Here over flow situation arises only when REAR is before the FRONT position.

e.g.

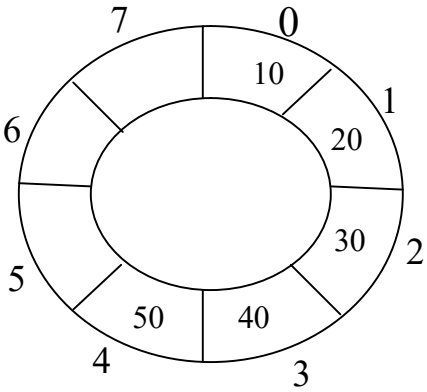
When Queue is Empty (or Underflow) the FRONT = NULL or -1 and when Queue is FULL (or Overflow) the FRONT = 0 and the REAR = N-1. When the queue having a single element the FRONT = REAR.

e.g. to enter elements into Queue

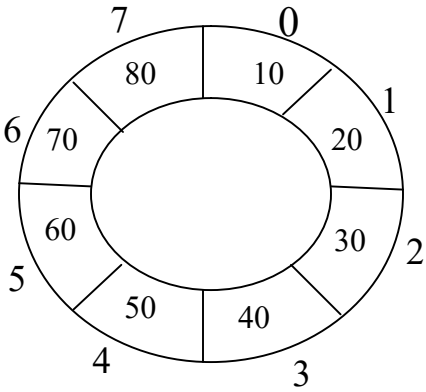
<u>insert 10,20,30,40,50</u> <u>insert 60,70,80</u>



FRONT= NULL
REAR = NULL

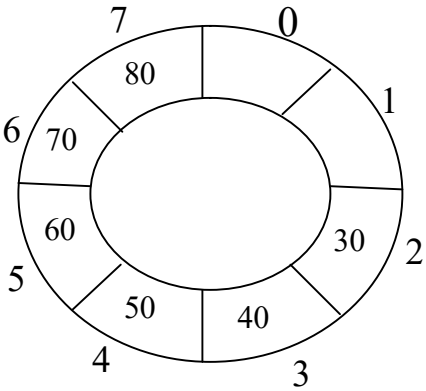


FRONT=0
REAR = 4



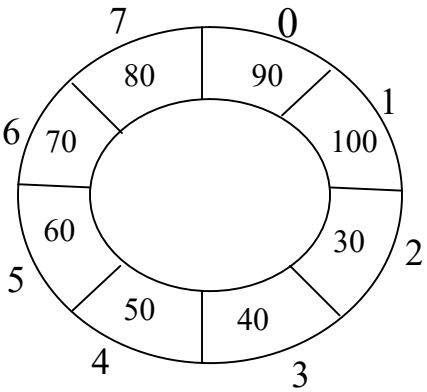
FRONT=0 Queue is full
REAR =7

<u>Deletion two elements</u>



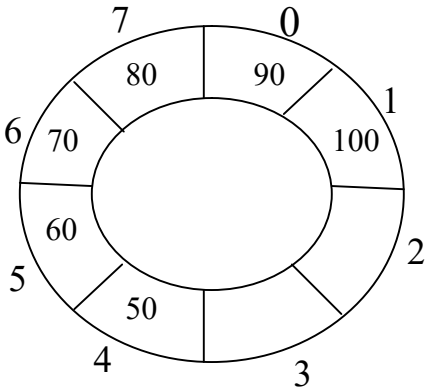
FRONT = 2
REAR = 7

<u>Insert one of 90,100</u>



FRONT = 2
REAR = 1

<u>Deletion two elements</u>



FRONT = 4
REAR = 1

1. If REAR=N, and Free Locations Exist, then REAR=0
2. If FRONT=N, and REAR is less than N then FRONT=0

1)

60			10	20	30	40	50

20	30	40					10

0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
R			F							R					F

During each insertion to find the next REAR position the formula is:

REAR=(REAR+1) % MAXSIZE

Similarly during each deletion to find the next FRONT position the formula is:

FRONT = (FRONT+1) % MAXSIZE

5.5.3 ALGORITHM FOR INSERTION IN A CIRCULAR QUEUE:

[Assume QUEUE [MAXSIZE] is an array representing a circular queue. FRONT and REAR identifies front and rear end of circular queue. ITEM is the new element to be inserted.]

QINSERT (QUEUE, FRONT, REAR, ITEM, MAXSIZE)

Step 1: **IF(FRONT=(REAR+1)%MAXSIZE)**
Display "Over Flow"
Exit.
Step 2: Else If FRONT=NULL, then
FRONT:=0
REAR:=0
Step 3: Else
REAR:=(REAR+1)% MAXSIZE
[End of IF]
Step 4: QUEUE [REAR]:= ITEM
Step 5: Exit.

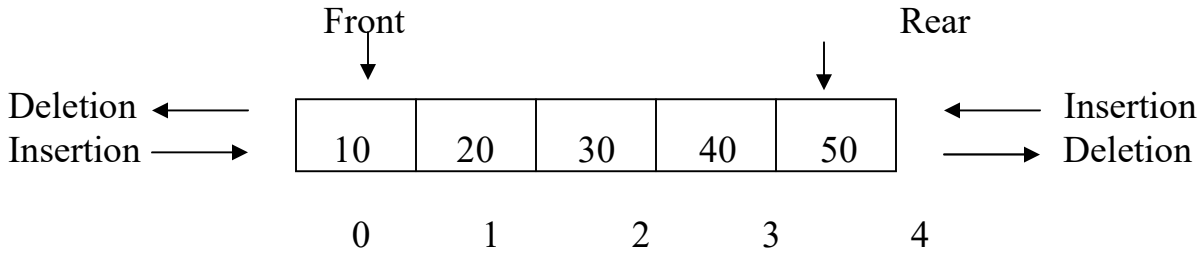
5.5.4 ALGORITHM FOR DELETION IN A CIRCULAR QUEUE:

QDELETION (QUEUE, FRONT, REAR)
Step 1: Let ITEM
Step 2: If FRONT=-1, Then

```
            Display "Queue is Empty or Underflow"
            Exit
        [End of IF]
Step 3: ITEM: = QUEUE [FRONT]
Step 4:     Display "Deleted Item", ITEM
Step 5:     If FRONT =REAR, Then
            FRONT: =-1, REAR=-1
Step 6:     Else
            FRONT :=( FRONT+1) % MAXSIZE
        [End of If]
Step 7:     Exit.
```

5.6 DOUBLE ENDED QUEUE (DEQUE)

Deque is another variation in queue, in which the elements can be added or removed at either ends but not the middle. In other words, it is a linear list in which insertion and deletion can be done at both the ends.



As insertion & deletion is allowed at both the ends of DEQUE, so following operations are possible in a deque.

1. Insertion at REAR End
2. Deletion at FRONT End
3. Insertion at FRONT End
4. Deletion at REAR End

5.6.1 Variations in Deque:

There are two types of DEQUE

1. Input Restricted Deque

2. Output Restricted Deque

5.6.1.1 ***Input Restricted Deque:***

It is a deque, which restricts insertion at only one end, but allows deletion at both ends of the list. So in deque, the following operations are possible.

- Deletion at FRONT end
- Deletion at REAR end
- Insertion at FRONT **or** REAR end

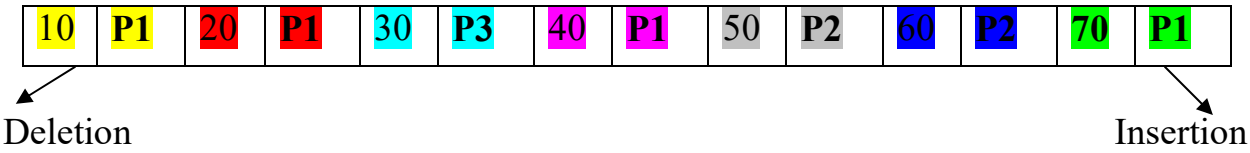
5.6.1.2 ***Output Restricted Deque:***

It is a deque, which restricts deletion at only one end, but allows insertion at both ends of the list. So in deque, the following operations are possible.

- Insertion at FRONT end
- Insertion at REAR end
- Deletion at FRONT **or** REAR end

5.6 Priority Queue:

A priority queue is a collection of elements such that each element has a priority level and according to the priority level, the elements are deleted. In the queue an element of higher priority is processed before any element of lower priority. When two or more elements having same priority levels then they are processed according to the order in which they are added to the queue.



5.8 Application Areas of Queue:

Queue is used in various aspects of operating systems. It is used to implement different CPU scheduling algorithm. Even in multiprogramming environment queue concept implemented to control various programs. Some applications are

simulation of traffic control system, CPU scheduling, multilevel queue scheduling and multilevel feedback queue scheduling and round robin technique in time sharing systems.

In multiprogramming, in order to maximize CPU utilization several processes are kept in memory at one time. When a process is waiting for I/O, the operating system assigns another process to the CPU. So the CPU utilization is maximized by switching the CPU among the processes. Here the CPU maintains four types of processes in individual queues. They are system process, interactive user process, batch process and student process.

System process maintains even driven scheduling, the interactive user process maintains shorter job first scheduling, batch process maintains round robin scheduling and student process maintains FCFS (First Come First Serve) scheduling.

The round robin scheduling is especially designed for time sharing systems. Here a small amount of time is called Time Slice. The processes are scheduled in a circular queue. Here the CPU starts service for 1st process for a specified time slice; afterwards CPU switches to 2nd process and so on. When a process finishes its execution then the process is deleted from the queue and when a new process exists then it is inserted at REAR of the queue.

The concept of queue also implemented in real life applications such as:

1) Control system of an automated factory having operator's console, which receives many messages, where messages will be in priority level to verify and act.

2) In a mail system, the emails are stored automatically in Inbox or bulk folders according to the priority.

3) Print Queue maintained by a Print server.

4) Railway ticket reservation and reservation status enquiry at touch screen.